

Page Pulse

Page Pulse audits any public URL and returns a clean report on HTTP health, response time, and on-page SEO/structure signals – in about a second, with no signup and no crawling delay.

Built for **Digital Heroes Training Task** ([Internship Programme](#)).

Table of contents

- [Deployment](#)
- [Project overview](#)
- [Architecture](#)
- [Technology choices](#)
- [Folder structure](#)
- [Installation & running locally](#)
- [Environment variables](#)
- [API documentation](#)
- [Testing](#)
- [Design decisions & trade-offs](#)
- [Error handling matrix](#)
- [Security](#)
- [Future improvements](#)

--

Deployment

Vercel:

1. Push this repo to GitHub.
2. Import the repo in Vercel, set the root directory to `frontend | backend`.
3. Set `VITE\_API\_BASE\_URL` to your deployed backend URL in case of deploying frontend.
4. Deploy – Vercel auto-detects the Vite framework preset.

Live URLs:

- Frontend: <https://digital-heros-tasks.vercel.app/>

- Backend: <https://kc-digital-heros-tasks.vercel.app/>
- GitHub: https://github.com/Khilesh-01/Digital_Heros_Tasks

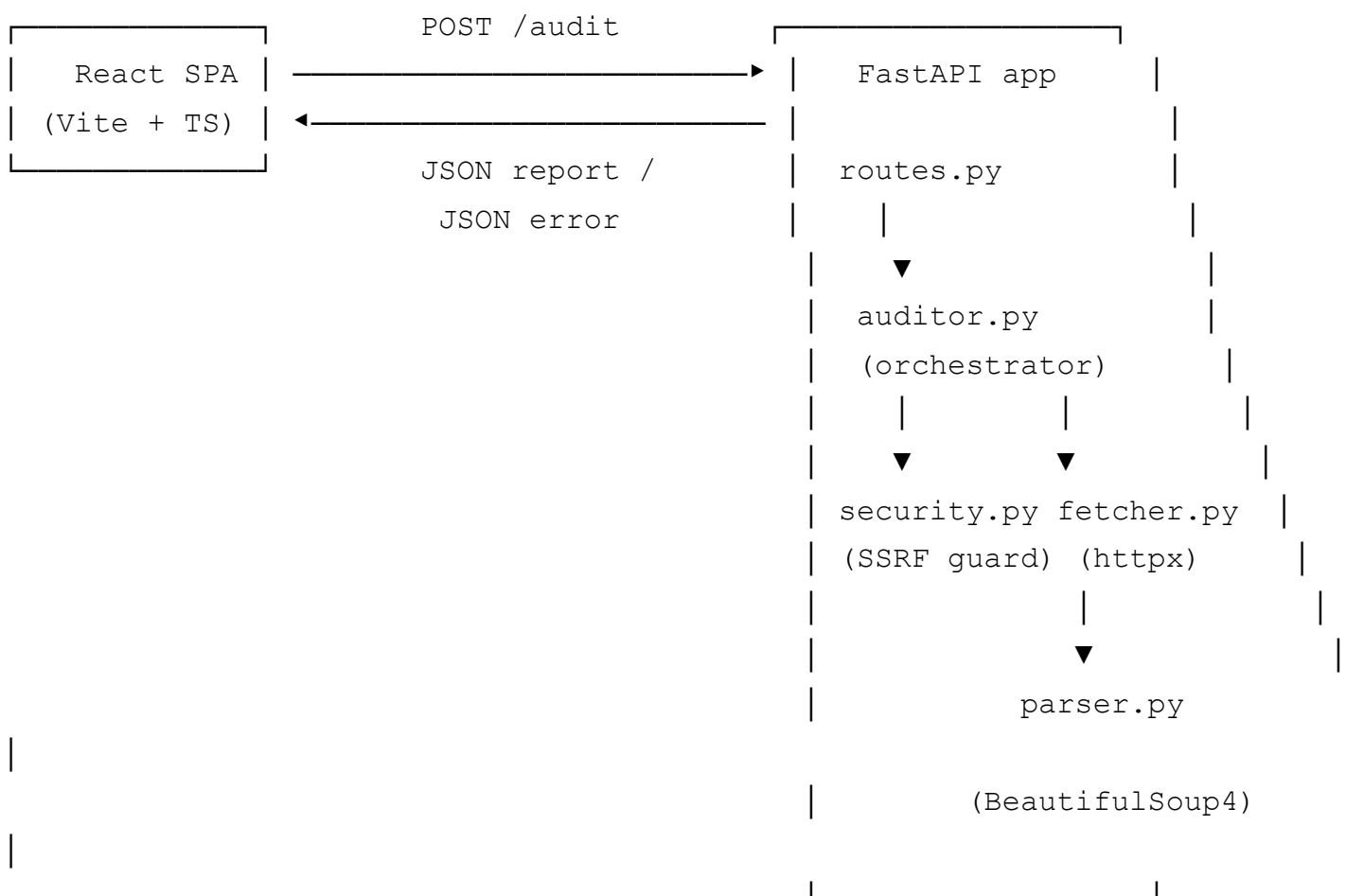
Project overview

Given a URL, Page Pulse:

1. Validates the URL and rejects obviously dangerous targets (private/loopback IPs).
2. Fetches the page with a bounded timeout, capped redirects, and a capped response size.
3. Parses the HTML for title, meta description, H1 count, image alt-text coverage, and an approximate visible word count.
4. Returns a single JSON report — or a clean, specific error if any step fails.

The frontend is a single-page React app: paste a URL, click **Audit page**, read the report. It never shows a stack trace, never hangs indefinitely, and degrades gracefully on every failure mode listed in the assignment brief.

Architecture



Request flow: `AuditForm` → `api/client.ts` (Axios) → `POST /audit` →
 Pydantic validates the body → `security.py` validates URL format and blocks

private/reserved network targets → `fetcher.py` performs the HTTP GET with a timeout, retry, and redirect budget → `parser.py` extracts every signal from the HTML → `auditor.py` assembles the `AuditReport` → FastAPI serializes it back to JSON.

Error flow: every failure mode (bad input, DNS failure, timeout, SSL error, non-HTML content, oversized response, upstream 4xx/5xx, or a truly unexpected exception) is raised as a typed `PagePulseError` subclass. A single set of exception handlers in `api/error\_handlers.py` translates every one of them into the same `{error\_code, message}` JSON shape and the correct HTTP status – so the frontend only ever needs one error-rendering path, and nothing ever leaks a Python traceback to the client.

Validation flow: Pydantic validates the request shape first (empty/too-long URL → `422 validation\_error`), then `security.py` validates URL structure and scheme (`422 invalid\_url`), then resolves the hostname and rejects private/loopback/reserved IPs (`422 disallowed\_target`) before any request is made to the target server.

Technology choices

Layer	Choice	Why
Backend	FastAPI + Python 3.12	Async-native, automatic OpenAPI docs, first-class Pydantic validation
HTTP client	<code>httpx(async)</code>	Native async support, fine-grained timeout/redirect controls, streaming reads for size-capping
Parsing	BeautifulSoup4 (<code>html.parser</code>)	Battle-tested, tolerant of malformed markup, no compiled dependencies to install
Frontend	React + Vite + TypeScript	Fast dev loop, typed contracts shared conceptually with the Pydantic models
HTTP client (FE)	Axios	Clean interceptor-friendly error handling vs. raw fetch
Testing	pytest + pytest-asyncio + FastAPI TestClient	Standard, fast, no external services required

Folder structure

```

page-pulse/
├─ backend/
│   └─ app/
│       └─ main.py                # FastAPI app, CORS, router wiring
│       └─ api/
│           └─ routes.py          # /audit, /health
│           └─ error\_handlers.py # centralized exception → JSON
translation
│   └─ core/
│       └─ config.py              # env-driven Settings (pydantic-
settings)
│   └─ exceptions.py             # typed domain exception hierarchy
│   └─ security.py               # URL validation + SSRF guard
│       └─ logging\_config.py
│   └─ models/
│       └─ schemas.py            # AuditRequest / AuditReport /
ErrorResponse
│   └─ services/
│       └─ fetcher.py            # httpx fetch with
retries/timeout/redirect caps
│   └─ parser.py                 # pure HTML → signals extraction
│       └─ auditor.py            # orchestrates fetch + parse → report
│   └─ tests/
│       └─ test\_parser.py        # happy path + 6 failure/edge cases
│       └─ test\_security.py     # URL validation + SSRF guard
│       └─ test\_api.py          # endpoint-level integration tests
│   └─ requirements.txt
│   └─ pytest.ini
│   └─ .env.example
├─ frontend/
│   └─ src/
│       └─ api/client.ts         # typed Axios client, normalized errors
│       └─ components/          # Header, PulseLine, AuditForm,
ReportCard,
│   └─ SkeletonLoader, ErrorAlert,
EmptyState, Toast, Footer
│   └─ styles/                  # tokens.css (design system) +
index.css
│   └─ types/audit.ts           # TS mirror of the backend contract
│       └─ App.tsx
└─ package.json

```

```
|   ├── vite.config.ts
|   ├── vercel.json
|   └── .env.example
└── docs/screenshots/
    ├── render.yaml
    └── .gitignore
```

Installation & running locally

Backend

```
cd backend
python -m venv .venv
source .venv/bin/activate    # Windows: .venv\\Scripts\\activate
pip install -r requirements.txt
cp .env.example .env         # optional - defaults already work
uvicorn app.main:app --reload
```

The API is now at <http://localhost:8000> . Interactive docs (Swagger UI) at <http://localhost:8000/docs> , ReDoc at <http://localhost:8000/redoc> .

Frontend

```
cd frontend
npm install
cp .env.example .env          # set VITE\_API\_BASE\_URL if not localhost:8000
npm run dev
```

The app is now at <http://localhost:5173> .

Environment variables

Backend ([backend/.env](#))

Variable	Default	Description
ENVIRONMENT	development	Free-text environment label, surfaced in logs

Variable	Default	Description
LOG_LEVEL	INFO	Python logging level
CORS_ORIGINS	http://localhost:5173,http://127.0.0.1:5173	Comma-separated allowed origins
REQUEST_TIMEOUT_SECONDS	10.0	Total time budget per fetch attempt
CONNECT_TIMEOUT_SECONDS	5.0	TCP/TLS connect budget
MAX_REDIRECTS	5	Redirect cap (protects against redirect loops)
MAX_RESPONSE_BYTES	5000000	Response size cap (5 MB)
USER_AGENT	PagePulseBot/1.0 (+https://digitalheroesco.com)	Sent on every outbound request
MAX_RETRIES	2	Retry attempts on timeout
RETRY_BACKOFF_SECONDS	0.5	Linear backoff between retries
ALLOW_PRIVATE_NETWORK_TARGETS	false	Set <code>true</code> only for local testing against localhost targets
ALLOWED_SCHEMES	http,https	Accepted URL schemes

Frontend(frontend/.env)

Variable	Default	Description
VITE_API_BASE_URL	http://localhost:8000	Base URL of the backend API

API documentation

Full interactive docs are auto-generated by FastAPI at `/docs` and `/redoc`.

Summary of the contract:

POST /audit

Request:

```
{ "url": "https://example.com" }
```

Success response — 200 OK :

```
{
  "url": "https://example.com/",
  "status": 200,
  "response\_time\_ms": 128,
  "title": "Example Domain",
  "meta\_description": null,
  "h1\_count": 1,
  "images\_without\_alt": 0,
  "word\_count": 28,
  "total\_images": 0,
  "canonical\_url": null,
  "og\_title": null,
  "favicon\_present": false,
  "language": null,
  "content\_size\_bytes": 1256,
  "seo\_score": 55
}
```

The first eight fields (`url` through `word\_count`) are the required assignment contract. Everything after `total\_images` is an additive bonus field — removing them would not break the contract.

Error response — status varies by failure (422 / 502 / 504):

```
{ "error\_code": "timeout", "message": "The request to the target page ti
```

GET /health

```
{ "status": "ok", "version": "1.0.0" }
```

Testing

```
cd backend
source .venv/bin/activate
pytest -v
```

24 tests across three files:

- `test\_parser.py` — happy path, missing title, missing meta description, malformed/unclosed HTML, empty HTML, word-count accuracy, images-without-alt counting (missing vs. blank `alt`), multiple `<h1>` tags.
- `test\_security.py` — valid URL, empty URL, missing scheme, unsupported scheme, missing host, localhost blocked, loopback IP blocked.
- `test\_api.py` — `/health`, full happy-path `/audit`, invalid URL, empty URL, timeout, DNS failure, non-HTML content, upstream 404, and a guarantee that unexpected exceptions never leak internal details to the client.

The HTTP fetch layer is mocked in API tests (`monkeypatch`), so the suite runs offline and deterministically.

Design decisions & trade-offs

1. Domain exceptions instead of inline HTTP error handling.

Every failure mode (`InvalidURLError`, `RequestTimeoutError`, `DNSResolutionError`, etc.) is a typed exception with its own `error\_code` / `http\_status`, raised deep inside services and translated to JSON in exactly one place (`api/error\_handlers.py`). *Trade-off*: more files/classes up front than a few `try/except` blocks in the route handler, but it means adding a new failure mode never risks an inconsistent error shape, and the route handler itself stays a two-line pass-through.

2. A best-effort SSRF guard, not a bulletproof one.

`core/security.py` blocks `localhost` and resolves the hostname to reject private/loopback/link-local/reserved IPs before fetching. *Trade-off*: this doesn't defend against DNS-rebinding attacks (resolving safely, then the target re-resolving to a private IP on the actual connection) — a production system would pin the resolved IP and connect to it directly. For a 3-4 hour assignment scope, the cheaper static-hostname + resolved-IP check was the right level of effort, and it's isolated in one module so it can be swapped for a stricter implementation later without touching anything else.

3. Streaming reads with a hard size cap, over `response.text` directly.

`fetcher.py` reads the response in chunks via `response.iter\_bytes()` and aborts once `MAX\_RESPONSE\_BYTES` is exceeded, rather than letting `httpx` buffer an arbitrarily large body first. *Trade-off*: slightly more code than `httpx.get(url).text`, but it means a multi-gigabyte or

infinite-stream response can never exhaust server memory — necessary for a tool whose whole job is fetching arbitrary, untrusted URLs.

Error handling matrix

Failure	HTTP status	<code>error_code</code>
Empty / malformed request body	422	<code>validation_error</code>
Invalid URL / bad scheme / missing host	422	<code>invalid_url</code>
Private/loopback/reserved network target	422	<code>disallowed_target</code>
Non-HTML content type	422	<code>unsupported_content_type</code>
Response exceeds size cap	422	<code>response_too_large</code>
DNS resolution failure	502	<code>dns_error</code>
Connection refused / reset	502	<code>connection_failed</code>
SSL certificate error	502	<code>ssl_error</code>
Too many redirects	502	<code>redirect_loop</code>
Upstream 4xx / 5xx	502	<code>upstream_http_error</code>
Timeout	504	<code>timeout</code>
Anything unanticipated	500	<code>internal_error</code> (message is always generic; details are logged server-side only)

Security

- URL scheme allow-list (`http` / `https` only).
- SSRF guard rejects localhost and private/loopback/reserved IP targets (see trade-offs above for its limits).
- Bounded request timeout, connect timeout, redirect count, and response size on every fetch.
- A descriptive `User-Agent` is always sent, and content-type is validated before parsing.
- Every error response is a fixed `{error\_code, message}` shape — never a raw exception message or stack trace.

- CORS is restricted to an explicit origin allow-list via `CORS\_ORIGINS`.

Future improvements

Given another day, in priority order:

1. **DNS-rebinding-safe fetching** — resolve once, connect to the pinned IP, and re-validate on every redirect hop (closes the gap noted in trade-off #2).
2. **Response caching** — a short-TTL cache keyed by URL to make repeated audits of the same page instant and reduce load on target sites.
3. **Historical trend view** — persist past audits (per URL) so the frontend can show response-time and score trends over time, not just a snapshot.
4. **Headless rendering option** — an opt-in mode using a headless browser for JS-heavy SPAs, since the current fetch-and-parse approach only sees server-rendered HTML.
5. **Rate limiting** — per-IP request throttling on `/audit` before this is exposed publicly at scale.